

**UNIVERSIDADE FEDERAL DO CEARÁ**

**CAMPUS QUIXADÁ**

**CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE**

**FRANCISCO WERLEY DA SILVA**

# APLICAÇÃO DE MÉTRICAS ESTRUTURAIS PARA AVALIAÇÃO DE MODULARIDADE EM SISTEMAS ORIENTADOS A OBJETOS: UM ESTUDO COMPARATIVO ENTRE ARQUITETURA EM CAMADAS E ARQUITETURA HEXAGONAL

**QUIXADÁ**

**2026**

FRANCISCO WERLEY DA SILVA

# APLICAÇÃO DE MÉTRICAS ESTRUTURAIS PARA AVALIAÇÃO DE MODULARIDADE EM SISTEMAS ORIENTADOS A OBJETOS: UM ESTUDO COMPARATIVO ENTRE ARQUITETURA EM CAMADAS E ARQUITETURA HEXAGONAL

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia de Software do Campus Quixadá da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia de Software.

Orientador: Prof. Dr. Emanuel Coutinho.

QUIXADÁ

2026

## RESUMO

A modularidade é um atributo importante para a qualidade estrutural de sistemas de software, pois está relacionada à forma como responsabilidades são distribuídas, dependências são estabelecidas e componentes podem ser compreendidos, modificados e evoluídos. Em sistemas orientados a objetos, decisões arquiteturais podem influenciar diretamente características como acoplamento, coesão e complexidade estrutural. Nesse contexto, estilos arquiteturais como a arquitetura em camadas e a arquitetura hexagonal propõem diferentes formas de organizar os elementos do sistema, podendo produzir efeitos distintos sobre sua modularidade. Diante disso, este trabalho tem como objetivo avaliar a modularidade de sistemas orientados a objetos por meio da aplicação de métricas estruturais em diferentes estilos arquiteturais. Para isso, será conduzido um estudo comparativo envolvendo dois sistemas desenvolvidos em contexto acadêmico: o SISO, estruturado segundo a arquitetura em camadas, e o Roomie, estruturado segundo a arquitetura hexagonal. A análise será realizada a partir de métricas extraídas por análise estática do código-fonte, considerando indicadores relacionados a acoplamento, coesão e complexidade estrutural. Espera-se que este trabalho contribua para a compreensão de como diferentes decisões arquiteturais podem se refletir em características estruturais mensuráveis de sistemas orientados a objetos. Além disso, busca-se fornecer evidências quantitativas que apoiem a análise da modularidade e auxiliem a adoção de práticas arquiteturais mais fundamentadas na Engenharia de Software.



**Palavras-chave**: Modularidade; Arquitetura de Software; Métricas de Software; Sistemas Orientados a Objetos.

## ABSTRACT

Modularity is an important attribute of software structural quality, as it is related to how responsibilities are distributed, dependencies are established, and components can be understood, modified, and evolved. In object-oriented systems, architectural decisions may directly influence characteristics such as coupling, cohesion, and structural complexity. In this context, architectural styles such as layered architecture and hexagonal architecture propose different ways of organizing system elements, which may produce distinct effects on software modularity. Therefore, this work aims to evaluate the modularity of object-oriented systems through the application of structural metrics across different architectural styles. To achieve this, a comparative study will be conducted involving two systems developed in an academic context: SISO, structured according to layered architecture, and Roomie, structured according to hexagonal architecture. The analysis will be based on metrics extracted through static analysis of the source code, considering indicators related to coupling, cohesion, and structural complexity. This work is expected to contribute to the understanding of how different architectural decisions may be reflected in measurable structural characteristics of object-oriented systems. Furthermore, it aims to provide quantitative evidence to support modularity analysis and assist the adoption of more grounded architectural practices in Software Engineering.



**Keywords**: Modularity; Software Architecture; Software Metrics; Object-Oriented Systems.

## LISTA DE FIGURAS

Figura 1 – Representação do padrão arquitetural em camadas . . . . . 12
Figura 2 – Representação do padrão arquitetural hexagonal . . . . . 14
Figura 3 – Fluxo metodológico da pesquisa . . . . . 35

## LISTA DE TABELAS

Tabela 1 – Caracterização dos sistemas analisados . . . . . 36
Tabela 2 – Cronograma de execução do TCC II . . . . . 42

## LISTA DE QUADROS

Quadro 1 – Síntese dos trabalhos relacionados . . . . . 33
Quadro 2 – Métricas utilizadas na avaliação da modularidade estrutural dos sistemas . 39

**SUMÁRIO**


**1 INTRODUÇÃO** . . . . . 9
**1.1 Objetivos** . . . . . 10
***1.1.1 Objetivo Geral*** . . . . . 10
***1.1.2 Objetivos Específicos*** . . . . . 10
**2 FUNDAMENTAÇÃO TEÓRICA** . . . . . 11
**2.1 Arquitetura de Software** . . . . . 11
***2.1.1 Arquitetura em Camadas*** . . . . . 12
***2.1.2 Arquitetura Hexagonal*** . . . . . 13
**2.2 Modularidade em Software** . . . . . 15
***2.2.1 Acoplamento*** . . . . . 16
***2.2.2 Coesão*** . . . . . 17
***2.2.3 Complexidade*** . . . . . 18
**2.3 Métricas de Software** . . . . . 18
**2.4 Métricas Estruturais em Sistemas Orientados a Objetos** . . . . . 19
***2.4.1 Coupling Between Object Classes (CBO)*** . . . . . 21
***2.4.2 Lack of Cohesion in Methods (LCOM)*** . . . . . 22
***2.4.3 Response For a Class (RFC)*** . . . . . 23
***2.4.4 Weighted Methods per Class (WMC)*** . . . . . 24
***2.4.5 Fan-In*** . . . . . 25
***2.4.6 Fan-Out*** . . . . . 26
**3 TRABALHOS RELACIONADOS** . . . . . 27
**3.1 Avaliação de Conectividade em Sistemas Orientados por Objetos** . . . . 27
**3.2 Os efeitos da arquitetura dirigida a eventos na modularidade de software** 28
**3.3 Comparação entre as arquiteturas de software monolítica e de microsserviços** . . . . . 29
**3.4 Análise comparativa entre arquiteturas de software: monolítica, modular e microsserviços** . . . . . 30
**3.5 Um estudo sobre a relação entre qualidade e arquitetura de software** . . 31
**3.6 Síntese e lacunas** . . . . . 32
**4 METODOLOGIA** . . . . . 35
**4.1 Objetos de Estudo** . . . . . 35

**4.2** **Fluxo Metodológico** . . . . . 36
***4.2.1*** ***Extração das Métricas Estruturais*** . . . . . 37
***4.2.2*** ***Processamento Automatizado dos Dados*** . . . . . 40
***4.2.3*** ***Análise dos Dados*** . . . . . 41
**4.3** **Cronograma de Execução** . . . . . 41
**REFERÊNCIAS BIBLIOGRÁFICAS** . . . . . 43

# 1 INTRODUÇÃO

A Engenharia de Software tem como objetivo o desenvolvimento de sistemas que atendam não apenas aos requisitos funcionais, mas também a atributos de qualidade que garantam sua sustentabilidade ao longo do tempo. Entre esses atributos, a manutenibilidade destaca-se como um dos mais relevantes, uma vez que grande parte dos custos de um sistema está associada à sua evolução e manutenção ao longo do ciclo de vida (Pressman, 2010). Nesse contexto, a modularidade é amplamente reconhecida como um fator fundamental para a melhoria da qualidade estrutural de sistemas de software, pois permite a decomposição do sistema em módulos menores e independentes, possibilitando que eles sejam modificados de forma isolada e sem que seja preciso conhecer profundamente a implementação de outros módulos (Parnas, 1972).

A modularidade está diretamente relacionada a conceitos como coesão e acoplamento. Sistemas com alta coesão e baixo acoplamento tendem a apresentar melhor organização estrutural, facilitando sua compreensão, teste e modificação (Yourdon; Constantine, 1978). Em sistemas orientados a objetos, essas características são fortemente influenciadas pelas decisões arquiteturais adotadas, uma vez que a arquitetura define a organização dos componentes e as relações de dependência entre eles (Bass *et al.*, 2012; Sommerville, 2011). Dessa forma, a escolha de um estilo arquitetural pode se refletir na qualidade estrutural do software.

Dentre os estilos arquiteturais amplamente utilizados, destacam-se a arquitetura em camadas e a arquitetura hexagonal. A arquitetura em camadas organiza o sistema em níveis hierárquicos, nos quais cada camada possui responsabilidades específicas, promovendo separação de responsabilidades (Fowler, 2003). Por outro lado, a arquitetura hexagonal, também conhecida como *Ports and Adapters*, propõe o isolamento do núcleo da aplicação em relação às dependências externas, favorecendo maior independência tecnológica e testabilidade (Cockburn, 2005). Ambos os estilos são discutidos na literatura e utilizados na prática, apresentando formas distintas de organizar responsabilidades e dependências no software.

Nesse cenário, métricas de software surgem como instrumentos fundamentais para a avaliação objetiva da qualidade estrutural. Métricas relacionadas à coesão e ao acoplamento permitem quantificar propriedades internas do código, fornecendo base para análise de diferentes soluções (Fenton; Bieman, 2015). Em particular, métricas estruturais para sistemas orientados a objetos, como as propostas por Chidamber e Kemerer (1994), são amplamente utilizadas para avaliar atributos internos do software, tais como acoplamento, coesão e complexidade, os quais

influenciam diretamente a modularidade e a organização estrutural dos componentes.

Apesar da existência de estudos que avaliam características estruturais de sistemas, os trabalhos relacionados analisados nesta pesquisa indicam que essas discussões têm sido conduzidas a partir de recortes arquiteturais distintos do proposto neste trabalho, como arquitetura dirigida a eventos, *REST*, monólitos e microsserviços (Lazzari; Farias, 2021; Cintra *et al.*, 2025; Vale, 2025). Assim, este trabalho propõe investigar a modularidade estrutural a partir de um recorte específico: a comparação quantitativa entre arquitetura em camadas e arquitetura hexagonal.

Este trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, abordando conceitos de arquitetura de software, modularidade e métricas estruturais; o Capítulo 3 discorre sobre alguns trabalhos relacionados; e, por fim, o Capítulo 4 descreve a metodologia que será adotada no TCC II.

## 1.1 Objetivos

Esta seção apresenta os objetivos deste trabalho.

### *1.1.1 Objetivo Geral*

Avaliar a modularidade de sistemas orientados a objetos por meio da aplicação de métricas estruturais, comparando um sistema organizado em arquitetura em camadas e um sistema organizado segundo a arquitetura hexagonal.

### *1.1.2 Objetivos Específicos*

- Caracterizar o perfil estrutural dos sistemas analisados por meio da extração de métricas estruturais;
- Comparar os indicadores de acoplamento, coesão e complexidade entre os diferentes estilos arquiteturais;
- Interpretar como os estilos arquiteturais adotados podem se refletir nos indicadores de modularidade observados.

# 2 FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica constitui a base conceitual necessária para compreensão dos temas abordados nesta pesquisa. Nesse contexto, este capítulo apresenta os principais conceitos relacionados à arquitetura de software, modularidade e qualidade estrutural de sistemas orientados a objetos. Também são discutidos os estilos arquiteturais analisados neste trabalho, bem como as métricas estruturais utilizadas para avaliar quantitativamente atributos relacionados ao acoplamento, à coesão e à complexidade dos sistemas estudados.

## 2.1 Arquitetura de Software

A arquitetura de software está relacionada à forma como um sistema é estruturado e compreendido em um nível mais alto de abstração. Segundo Bass *et al.* (2012), a arquitetura de um sistema corresponde ao conjunto de estruturas necessárias para raciocinar sobre ele, incluindo seus elementos de software, as relações estabelecidas entre esses elementos e as propriedades associadas a ambos.

Essa definição evidencia que a arquitetura não é composta por uma única estrutura, mas por diferentes estruturas que permitem analisar o sistema sob perspectivas distintas. De acordo com Bass *et al.* (2012), uma estrutura pode ser compreendida como um conjunto de elementos ligados por algum tipo de relação. Assim, a arquitetura permite observar como os elementos do sistema se organizam e como se relacionam, omitindo detalhes internos que não são relevantes para a compreensão da estrutura geral.

De forma complementar, Fowler (2003) destaca que o termo arquitetura não possui uma definição única amplamente aceita, mas geralmente envolve dois aspectos recorrentes: a decomposição do sistema em partes de alto nível e as decisões consideradas difíceis de modificar posteriormente. Nesse sentido, a arquitetura representa não apenas a organização estrutural do software, mas também um conjunto de decisões que influenciam a forma como o sistema pode ser compreendido, mantido e evoluído.

Em determinados contextos, os elementos arquiteturais são organizados de maneiras recorrentes para solucionar problemas comuns de projeto. Essas composições, descritas na literatura como padrões arquiteturais, fornecem estratégias reutilizáveis para estruturar sistemas de software (Bass *et al.*, 2012; Fowler, 2003). Entre os estilos relevantes para esta pesquisa destacam-se a arquitetura em camadas e a arquitetura hexagonal, que serão discutidas nas

subseções seguintes.

### *2.1.1 Arquitetura em Camadas*

A arquitetura em camadas é um padrão arquitetural no qual os elementos do sistema são organizados em níveis de responsabilidade, chamados camadas. Segundo Bass *et al.* (2012), esse padrão emerge quando a relação de uso entre os elementos de software ocorre de forma unidirecional, de modo que uma camada utiliza os serviços fornecidos pelas camadas inferiores.

De acordo com Bass *et al.* (2012), uma camada corresponde a um conjunto coerente de funcionalidades relacionadas. Em uma estrutura estritamente em camadas, cada camada deve utilizar apenas os serviços oferecidos pela camada imediatamente inferior. Essa organização estabelece uma ordem clara de dependência entre as partes do sistema, favorecendo a separação de responsabilidades e a definição de interfaces públicas entre os níveis da aplicação.

O padrão em camadas tem como objetivo permitir que diferentes partes do sistema sejam desenvolvidas e evoluídas com menor interação direta entre si. Para isso, o software é segmentado em unidades que agrupam módulos responsáveis por oferecer um conjunto coeso de serviços. A Figura 1 apresenta uma representação simplificada desse padrão, na qual a camada superior pode utilizar os serviços da camada imediatamente inferior, mantendo uma relação de dependência orientada de cima para baixo.

Figura 1 – Representação do padrão arquitetural em camadas




Fonte: Elaborado pelo autor (2026), com base em Bass *et al.* (2012).

Na prática, existem variações desse padrão. Embora a estrutura estrita permita apenas o uso da camada imediatamente inferior, Bass *et al.* (2012) observam que alguns sistemas permitem o acesso a camadas inferiores não adjacentes, situação conhecida como *layer bridging*. Entretanto, a ocorrência frequente desse tipo de acesso pode comprometer benefícios associados ao padrão, como modificabilidade e portabilidade, pois aumenta o acoplamento entre partes que

deveriam permanecer separadas.

Embora a arquitetura em camadas seja definida principalmente pela separação de responsabilidades e pela relação de uso entre camadas, sua implementação pode empregar mecanismos de abstração para reduzir dependências diretas entre seus elementos. Conforme discutido por Bass *et al.* (2012), as camadas expõem seus serviços por meio de interfaces públicas, permitindo que uma camada utilize funcionalidades fornecidas por outra sem depender de seus detalhes internos de implementação. Assim, o uso de interfaces entre camadas não descaracteriza a arquitetura em camadas, mas representa uma forma de controlar o acoplamento entre seus elementos.

Dessa forma, a arquitetura em camadas organiza o software por meio de níveis de responsabilidade e relações de dependência controladas. Essa característica torna o padrão relevante para estudos sobre modularidade, uma vez que a forma como as camadas são estruturadas influencia diretamente o acoplamento entre os componentes e a separação de responsabilidades no sistema.

### *2.1.2 Arquitetura Hexagonal*

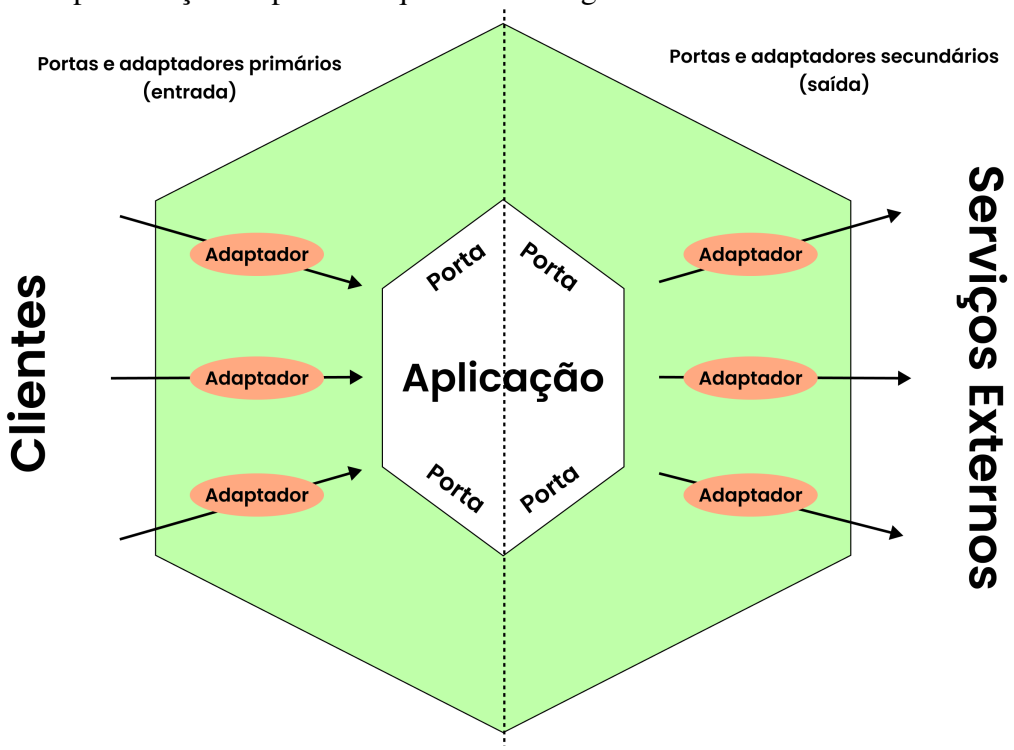
A arquitetura hexagonal, também conhecida como *Ports and Adapters*, foi proposta por Cockburn (2005) com o objetivo de permitir que uma aplicação seja desenvolvida e testada de forma isolada em relação às tecnologias externas utilizadas em sua execução. Nesse modelo, a aplicação deve ser capaz de interagir com diferentes mecanismos externos, como interfaces de usuário, bancos de dados, testes automatizados, scripts ou outros sistemas, sem que suas regras internas dependam diretamente dessas tecnologias (Cockburn, 2005).

Segundo Cockburn (2005), a motivação central desse padrão está relacionada à necessidade de evitar o acoplamento entre a lógica da aplicação e os elementos externos ao sistema. O autor destaca que problemas semelhantes ocorrem tanto quando regras de negócio são incorporadas à interface com o usuário quanto quando a lógica da aplicação passa a depender diretamente de bancos de dados ou serviços externos. Nesses casos, alterações tecnológicas podem afetar partes centrais do sistema, dificultando testes e a substituição de mecanismos externos.

A solução proposta pela arquitetura hexagonal consiste em estabelecer uma separação entre o interior e o exterior da aplicação. O núcleo da aplicação comunica-se com elementos externos por meio de portas, que representam interfaces ou protocolos de interação. Os adapta-

dores, por sua vez, são responsáveis por converter chamadas, mensagens ou sinais específicos de uma tecnologia externa para uma forma compreensível pela aplicação, e vice-versa (Cockburn, 2005). A Figura 2 apresenta uma representação desse padrão.

Figura 2 – Representação do padrão arquitetural hexagonal




Fonte: Elaborado pelo autor (2026), com base em Cockburn (2005).

Nesse padrão, o termo “hexagonal” não indica que a aplicação deve possuir exatamente seis lados ou seis interfaces. De acordo com Cockburn (2005), o uso do hexágono busca representar visualmente a existência de múltiplas portas ao redor da aplicação, destacando a separação entre o núcleo interno e os elementos externos. Assim, diferentes tipos de adaptadores podem ser conectados às portas da aplicação, permitindo substituir uma interface gráfica por testes automatizados ou trocar um banco de dados real por uma implementação em memória, sem alterar as regras internas do sistema (Cockburn, 2005).

Em implementações orientadas a objetos, as portas podem ser representadas por interfaces de programação que definem os serviços oferecidos ou requeridos pelo núcleo da aplicação. Os adaptadores implementam essas portas e conectam a aplicação a tecnologias específicas, como interfaces gráficas, *APIs*, mecanismos de persistência ou serviços externos (Cockburn, 2005). Essa organização favorece a independência tecnológica, pois as regras internas da aplicação permanecem separadas dos detalhes de infraestrutura e integração.

A arquitetura hexagonal diferencia-se da arquitetura em camadas por enfatizar a rela-

ção entre o interior e o exterior da aplicação, em vez de representar o sistema apenas como uma sequência vertical de camadas (Cockburn, 2005). Enquanto a arquitetura em camadas organiza o software em níveis hierárquicos de responsabilidade, a arquitetura hexagonal concentra-se na proteção do núcleo da aplicação contra dependências tecnológicas externas. Essa característica torna o padrão relevante para estudos sobre modularidade, uma vez que a forma como portas e adaptadores são definidos pode influenciar as dependências entre componentes e a separação entre regras de negócio e infraestrutura.

## 2.2 Modularidade em Software

A modularidade está diretamente relacionada à forma como um sistema é dividido em partes menores e mais gerenciáveis (Parnas, 1972). Em sistemas modulares, as funcionalidades são organizadas em unidades relativamente independentes, permitindo que cada módulo desempenhe responsabilidades específicas e possa ser compreendido, desenvolvido e modificado com menor impacto sobre os demais componentes do sistema.

Um dos trabalhos pioneiros sobre modularização foi apresentado por Parnas (1972), que argumenta que a decomposição de sistemas em módulos deve ser realizada de forma a minimizar a propagação de mudanças entre diferentes partes do software. Segundo o autor, uma estrutura modular adequada reduz a complexidade do desenvolvimento e facilita atividades de manutenção, evolução e correção de defeitos.

De acordo com Sommerville (2011), sistemas complexos podem ser organizados em componentes com responsabilidades específicas, favorecendo sua compreensão e reduzindo a complexidade do desenvolvimento. Essa decomposição também contribui para a reutilização de funcionalidades, uma vez que componentes bem definidos podem ser empregados em diferentes contextos com menor necessidade de modificações.

A importância da modularidade está associada à capacidade de controlar a complexidade inerente ao desenvolvimento de software. Segundo Pressman (2010), sistemas de grande porte tornam-se progressivamente mais difíceis de compreender e modificar quando apresentam dependências excessivas entre seus componentes. Nesse contexto, a modularização surge como uma estratégia para organizar responsabilidades e limitar o impacto das mudanças realizadas ao longo do ciclo de vida do software.

Embora o conceito de modularidade seja amplamente discutido na literatura, sua avaliação objetiva não é uma tarefa trivial. Conforme discutido por Fenton e Bieman (2015),

atributos relacionados à qualidade interna do software frequentemente apresentam natureza abstrata e, por esse motivo, podem ser avaliados por meio de indicadores quantitativos. Dessa forma, características associadas à modularidade são frequentemente investigadas por intermédio de métricas estruturais capazes de mensurar propriedades observáveis do código-fonte.

Entre os atributos frequentemente associados à avaliação da modularidade e da qualidade estrutural de sistemas orientados a objetos destacam-se o acoplamento, a coesão e a complexidade estrutural (Pressman, 2010; Fenton; Bieman, 2015). Assim, esses atributos serão discutidos nas subseções seguintes, pois permitem analisar diferentes aspectos da organização estrutural do software.

### *2.2.1 Acoplamento*

O acoplamento representa o grau de dependência existente entre módulos, componentes ou classes de um sistema de software. Em sistemas orientados a objetos, essa dependência ocorre quando uma classe estabelece relacionamentos com outras classes para utilizar seus serviços, trocar dados ou acessar elementos necessários à implementação de suas responsabilidades (Pressman, 2010).

Segundo Pressman (2010), à medida que componentes se tornam mais interdependentes, o grau de acoplamento aumenta. Como consequência, alterações realizadas em um componente tendem a produzir impactos em outros componentes relacionados.

Embora algum nível de acoplamento seja inevitável em sistemas de software, uma vez que seus componentes precisam se comunicar para executar suas funcionalidades, Pressman (2010) destaca que o projeto deve buscar reduzi-lo sempre que possível. Sistemas com baixo acoplamento favorecem a independência entre módulos, limitam a propagação de mudanças e facilitam a reutilização dos componentes.

O conceito de baixo acoplamento também está relacionado ao princípio da ocultação de informações (*information hiding*) proposto por Parnas (1972). Segundo esse princípio, os módulos devem expor apenas as informações estritamente necessárias para sua utilização, preservando seus detalhes internos de implementação. Dessa forma, modificações realizadas em um componente tendem a produzir menor impacto sobre os demais elementos do sistema.

O acoplamento constitui, portanto, um atributo importante para a organização de sistemas modulares, uma vez que influencia a independência entre componentes e a propagação de mudanças ao longo do software. Entretanto, a qualidade estrutural de um sistema não depende

apenas das relações estabelecidas entre seus módulos, mas também da organização interna de cada um deles. Esse aspecto é representado pelo conceito de coesão, discutido na subseção seguinte.

### *2.2.2 Coesão*

A coesão refere-se ao grau de relacionamento existente entre os elementos internos de um módulo ou classe. Em sistemas orientados a objetos, um componente é considerado coeso quando encapsula atributos e operações fortemente relacionados entre si e direcionados a um mesmo propósito (Pressman, 2010).

Segundo Pressman (2010), a coesão constitui uma característica desejável em projetos de software, pois componentes cujos elementos estão fortemente relacionados tendem a ser mais simples de implementar, testar e manter. Em contrapartida, componentes que agrupam funcionalidades pouco relacionadas costumam apresentar maior complexidade e dificultam atividades de manutenção e evolução do sistema.

De acordo com Bass *et al.* (2012), a coesão está diretamente relacionada ao grau de associação entre as responsabilidades atribuídas a um módulo. Os autores descrevem esse atributo como uma medida da "unidade de propósito" (*unity of purpose*) de um componente, indicando que módulos altamente coesos tendem a concentrar responsabilidades relacionadas, reduzindo a probabilidade de que uma mudança em uma responsabilidade afete funcionalidades distintas.

Assim como o acoplamento descreve as dependências existentes entre diferentes componentes, a coesão concentra-se na organização interna de cada módulo. Dessa forma, sistemas modulares buscam simultaneamente reduzir as dependências entre componentes e concentrar responsabilidades relacionadas em unidades bem definidas.

Nesse contexto, a coesão constitui um atributo relevante para avaliação da modularidade de sistemas de software, complementando a análise do acoplamento ao considerar a forma como responsabilidades e funcionalidades são organizadas no interior de cada componente. Além disso, a avaliação da qualidade estrutural também envolve a análise da complexidade resultante da organização e das relações estabelecidas entre os elementos do sistema.

### *2.2.3 Complexidade*

Neste trabalho, a complexidade é compreendida sob uma perspectiva estrutural, estando relacionada à dificuldade de compreender a organização de um sistema de software e as relações estabelecidas entre seus elementos (Pressman, 2010; Fenton; Bieman, 2015). Em sistemas orientados a objetos, essa complexidade pode ser observada a partir da forma como classes, métodos e dependências se organizam e interagem ao longo da estrutura do sistema.

Segundo Pressman (2010), existem diferentes formas de compreender a complexidade de software. No contexto de projetos orientados a objetos, a complexidade pode ser analisada a partir de características estruturais, especialmente pela forma como as classes se relacionam entre si. Dessa maneira, sistemas com maior quantidade de elementos interdependentes tendem a exigir maior esforço de compreensão, manutenção e evolução.

De forma complementar, Fenton e Bieman (2015) associam a complexidade estrutural à complexidade das conexões entre os elementos de um modelo de sistema. Para os autores, essa complexidade depende do número de ligações existentes entre os elementos, indicando que sistemas com maior quantidade de relações estruturais tendem a apresentar maior complexidade interna. Assim, a análise da complexidade estrutural permite observar não apenas a quantidade de elementos presentes no sistema, mas também como esses elementos estão conectados.

No contexto desta pesquisa, a complexidade é analisada a partir de propriedades observáveis no código-fonte dos sistemas. Dessa forma, não se pretende avaliar a complexidade algorítmica ou o desempenho das aplicações, mas sim características relacionadas à organização das classes, à quantidade de operações e ao potencial de interação entre os componentes.

Assim, a complexidade estrutural complementa os conceitos de acoplamento e coesão na avaliação da qualidade estrutural do software. Enquanto o acoplamento considera as dependências entre classes e a coesão considera a organização interna de cada módulo, a complexidade contribui para compreender o esforço necessário para analisar, modificar e evoluir os elementos estruturais do sistema.

## 2.3 Métricas de Software

A mensuração é uma atividade importante para analisar características de sistemas de software de forma objetiva. Segundo Fenton e Bieman (2015), medição é o processo pelo qual números ou símbolos são atribuídos a atributos de entidades do mundo real, de acordo com

regras claramente definidas. Dessa forma, o processo de medição não avalia uma entidade de maneira genérica, mas atributos específicos que se deseja observar.

No contexto da Engenharia de Software, métricas permitem representar quantitativamente características de sistemas, componentes ou processos. De acordo com Sommerville (2011), uma métrica de software corresponde a uma característica de um sistema de software, de sua documentação ou de seu processo de desenvolvimento que pode ser medida objetivamente. Entre essas características, podem ser observados aspectos relacionados ao tamanho, à complexidade, à ocorrência de falhas ou ao esforço necessário para desenvolver componentes.

As medições de um sistema de software podem ser utilizadas para apoiar a análise de atributos de qualidade. Segundo Sommerville (2011), características de componentes podem ser medidas e utilizadas para avaliar atributos mais gerais do sistema. Nesta pesquisa, esse princípio é aplicado à análise de características estruturais de classes, como dependências, coesão e complexidade, permitindo observar diferenças na organização interna dos sistemas estudados.

Embora forneçam valores objetivos sobre características observáveis do software, métricas não devem ser interpretadas isoladamente como medidas absolutas de qualidade. Fenton e Bieman (2015) destacam que toda ação de medição deve estar associada a um objetivo ou necessidade claramente definida. Assim, os valores obtidos por métricas precisam ser analisados de acordo com o contexto do sistema e com o propósito da avaliação.

No contexto deste trabalho, as métricas de software são utilizadas como indicadores quantitativos para apoiar a análise da modularidade e da qualidade estrutural dos sistemas estudados. Como o foco está em sistemas orientados a objetos, interessam especialmente métricas extraídas do código-fonte, capazes de fornecer indicadores sobre características estruturais relacionadas a classes, dependências, coesão e complexidade. Essas métricas são discutidas na seção seguinte.

## 2.4 Métricas Estruturais em Sistemas Orientados a Objetos

Em sistemas orientados a objetos, a estrutura do software é organizada principalmente a partir de classes, objetos, métodos, interfaces e relações entre esses elementos. Fenton e Bieman (2015) destacam que projetos e implementações orientados a objetos são construídos com base em classes, que definem conjuntos de objetos instanciáveis e agrupam métodos relacionados ao comportamento desses objetos. Além disso, linguagens orientadas a objetos podem incluir mecanismos como interfaces, herança e ligação dinâmica, que aumentam a expressividade dos

desenvolvedores, mas também podem tornar projetos e implementações mais complexos.

Essa forma de organização torna necessária a utilização de métricas capazes de representar características próprias do paradigma orientado a objetos. Chidamber e Kemerer (1994) argumentam que métricas desenvolvidas para abordagens tradicionais de desenvolvimento não se ajustam diretamente a conceitos como classes, herança, encapsulamento e troca de mensagens. Por esse motivo, os autores propõem um conjunto de métricas voltado especificamente à avaliação de projetos orientados a objetos.

A classe ocupa papel central nesse tipo de avaliação. De acordo com Pressman (2010), a classe é uma unidade fundamental em sistemas orientados a objetos, pois encapsula dados e operações, pode participar de hierarquias de herança e colabora com outras classes para realizar as funcionalidades do sistema. Dessa forma, métricas aplicadas no nível de classe podem fornecer informações relevantes sobre características estruturais do software, como acoplamento, coesão, complexidade e comunicação entre componentes.

De forma semelhante, Chidamber e Kemerer (1994) afirmam que o projeto de classes é central no paradigma orientado a objetos, razão pela qual as métricas propostas pelos autores foram definidas para capturar aspectos estruturais relacionados às classes e às relações estabelecidas entre elas. Essas métricas são consideradas estáticas, pois podem ser coletadas a partir da estrutura do projeto ou da implementação, sem a necessidade de executar o sistema.

A suíte CK, proposta por Chidamber e Kemerer, é composta por seis métricas orientadas a classes: *Weighted Methods per Class* (WMC), *Depth of Inheritance Tree* (DIT), *Number of Children* (NOC), *Coupling Between Object Classes* (CBO), *Response For a Class* (RFC) e *Lack of Cohesion in Methods* (LCOM) (Chidamber; Kemerer, 1994). Essas métricas abordam diferentes dimensões do projeto orientado a objetos, incluindo complexidade interna da classe, uso de herança, dependências entre classes, conjunto de respostas possíveis e coesão entre métodos.

Além das métricas da suíte CK, medidas relacionadas ao fluxo de dependências entre componentes também são utilizadas para analisar a estrutura de sistemas de software. Nesse contexto, *Fan-In* e *Fan-Out* são discutidas na literatura como medidas associadas, respectivamente, à quantidade de elementos que dependem de um componente e à quantidade de elementos dos quais um componente depende (Yourdon; Constantine, 1978; Sommerville, 2011). Essas métricas complementam a análise de acoplamento ao permitir observar a direção e a concentração das dependências no sistema.

No contexto desta pesquisa, o interesse recai sobre métricas relacionadas à modularidade e à qualidade estrutural dos sistemas estudados. Por esse motivo, são consideradas métricas associadas principalmente ao acoplamento, à coesão e à complexidade estrutural das classes. As subseções seguintes apresentam as métricas utilizadas no estudo, incluindo CBO, LCOM, RFC, WMC, *Fan-In* e *Fan-Out*, discutindo sua relação com os atributos estruturais analisados.

É importante destacar que algumas métricas orientadas a objetos podem apresentar variações de definição e implementação na literatura e em ferramentas de análise estática. A própria definição da WMC, por exemplo, considera a soma das complexidades dos métodos de uma classe, mas Chidamber e Kemerer (1994) indicam que a forma específica de medir essa complexidade pode depender da implementação adotada. Assim, neste capítulo as métricas são apresentadas em seu significado conceitual e em sua relação com os atributos analisados, enquanto a forma específica de operacionalização adotada pela ferramenta utilizada nesta pesquisa é descrita na Metodologia.

### *2.4.1 Coupling Between Object Classes (CBO)*

A métrica *Coupling Between Object Classes* (CBO) tem como objetivo representar o grau de acoplamento existente entre uma classe e as demais classes do sistema. Segundo Chidamber e Kemerer (1994), o CBO de uma classe corresponde ao número de outras classes às quais ela está acoplada. Essa relação ocorre quando métodos de uma classe utilizam métodos ou variáveis de instância definidos em outra classe.

Em sistemas orientados a objetos, valores elevados de CBO indicam maior dependência entre classes. De acordo com Sommerville (2011), classes são consideradas acopladas quando métodos de uma classe utilizam métodos ou variáveis definidos em outra. Assim, quanto maior o valor de CBO, maior tende a ser a dependência da classe em relação a outros elementos do sistema, aumentando a possibilidade de que alterações em uma classe produzam impactos em outras partes do programa.

A interpretação dessa métrica também está associada à modularidade e à reutilização. Pressman (2010) observa que, à medida que o valor de CBO aumenta, a reutilização de uma classe tende a diminuir, pois a classe passa a depender de um número maior de colaborações externas. Além disso, altos valores de CBO podem dificultar modificações e testes, uma vez que mudanças em classes acopladas podem exigir maior atenção aos efeitos propagados pelo sistema.

Apesar de sua relevância, o CBO deve ser interpretado com cautela. Fenton e Bieman (2015) destacam que a métrica contabiliza o número de classes às quais uma classe está acoplada, e não necessariamente o número total de conexões existentes entre elas. Dessa forma, duas classes podem apresentar o mesmo valor de CBO mesmo que possuam intensidades diferentes de interação. Assim, o CBO deve ser compreendido como um indicador do alcance do acoplamento de uma classe, e não como uma medida completa de todas as dependências internas existentes no sistema.

Neste trabalho, o CBO será interpretado como um indicador do acoplamento estrutural das classes, permitindo observar quais partes dos sistemas concentram maior número de dependências com outros elementos do código. Dessa forma, o CBO contribui para investigar como a arquitetura em camadas e a arquitetura hexagonal se refletem nas dependências estruturais observadas no código-fonte.

### *2.4.2 Lack of Cohesion in Methods (LCOM)*

A métrica *Lack of Cohesion in Methods* (LCOM) indica a falta de coesão interna de uma classe. Diferentemente de métricas que avaliam dependências entre classes, a LCOM concentra-se na relação entre os métodos de uma classe e os atributos ou variáveis de instância utilizados por esses métodos.

Segundo Fenton e Bieman (2015), a coesão de uma classe pode ser caracterizada pelo grau de relacionamento entre seus métodos locais e suas variáveis de instância. Nesse sentido, métodos que acessam ou modificam atributos em comum tendem a indicar maior proximidade funcional dentro da classe. Por outro lado, quando os métodos utilizam conjuntos distintos de atributos, a classe pode apresentar menor coesão interna.

A LCOM é considerada uma métrica inversa de coesão, pois valores mais altos indicam menor coesão entre os métodos da classe (Fenton; Bieman, 2015). Essa interpretação também é discutida por Pressman (2010), ao destacar que valores elevados de LCOM podem indicar maior complexidade no projeto da classe. De modo geral, busca-se manter a coesão elevada, o que corresponde a valores mais baixos para métricas de falta de coesão.

Entretanto, a definição e a utilidade da LCOM devem ser interpretadas com cautela. Sommerville (2011) observa que essa métrica existe em diferentes variações e que seu valor tem sido objeto de debate na literatura. De forma semelhante, Fenton e Bieman (2015) destacam limitações associadas à formulação original da métrica, especialmente por não ser normalizada.

Assim, a LCOM deve ser compreendida como um indicador relacionado à coesão interna da classe, e não como uma medida absoluta da qualidade de seu projeto.

Nesse trabalho, a LCOM é considerada como uma métrica relacionada à coesão interna das classes. A forma específica de operacionalização adotada neste trabalho é apresentada na Metodologia, juntamente com as demais métricas utilizadas na análise estrutural dos sistemas.

### *2.4.3 Response For a Class (RFC)*

A métrica *Response For a Class* (RFC) representa o tamanho do conjunto de respostas possíveis de uma classe. De acordo com Chidamber e Kemerer (1994), o conjunto de respostas de uma classe corresponde ao conjunto de métodos que podem ser potencialmente executados em resposta a uma mensagem recebida por um objeto dessa classe.

Esse conjunto inclui tanto os métodos definidos na própria classe quanto os métodos externos que podem ser invocados por seus métodos locais. Nesse sentido, Fenton e Bieman (2015) descrevem a RFC como a soma entre o número de métodos locais e o número de métodos externos chamados por esses métodos. Dessa forma, a métrica permite observar não apenas a quantidade de operações diretamente disponíveis em uma classe, mas também o potencial de interação dessa classe com outros elementos do sistema.

Valores elevados de RFC indicam que uma classe possui um conjunto maior de comportamentos potencialmente executáveis em resposta a uma chamada. Pressman (2010) observa que, à medida que a RFC aumenta, também tende a aumentar o esforço necessário para testar a classe, pois cresce o número de métodos que podem participar das sequências de execução. De forma semelhante, Sommerville (2011) relaciona valores mais altos de RFC a maior complexidade da classe, uma vez que mais métodos podem ser acionados a partir de uma mensagem recebida.

Apesar de sua utilidade, a RFC deve ser interpretada como um indicador do tamanho do conjunto de respostas de uma classe, e não como uma representação completa de seu comportamento em tempo de execução. Conforme discutido por Fenton e Bieman (2015), a métrica contabiliza os métodos presentes no conjunto de respostas, mas não necessariamente o número total de chamadas realizadas ou a frequência com que essas chamadas ocorrem. Assim, classes com valores semelhantes de RFC podem apresentar diferentes intensidades de interação interna ou externa.

No contexto desta pesquisa, a RFC é utilizada como indicador relacionado à com-

plexidade estrutural e ao potencial de interação das classes. A análise dessa métrica permite observar se os estilos arquiteturais estudados estão associados a classes com maior ou menor conjunto de respostas possíveis, contribuindo para a comparação da organização estrutural dos sistemas analisados.

### *2.4.4 Weighted Methods per Class (WMC)*

A métrica *Weighted Methods per Class* (WMC) está relacionada à quantidade e à complexidade dos métodos definidos em uma classe. Segundo Chidamber e Kemerer (1994), a WMC é calculada a partir da soma dos pesos atribuídos aos métodos de uma classe, em que cada peso representa um fator de complexidade associado ao respectivo método.

A interpretação da WMC depende da forma como os pesos dos métodos são definidos. Fenton e Bieman (2015) destacam que, quando todos os métodos recebem peso igual a 1, a WMC passa a representar uma medida de tamanho da classe, correspondendo ao número de métodos nela definidos. Dessa forma, a métrica pode ser compreendida tanto como uma medida de complexidade quanto como uma medida de tamanho, dependendo da estratégia adotada para atribuição dos pesos.

De acordo com Pressman (2010), o número de métodos e a complexidade desses métodos são indicadores razoáveis do esforço necessário para implementar e testar uma classe. O autor também observa que, à medida que o número de métodos cresce, a classe tende a se tornar mais específica da aplicação, o que pode limitar seu potencial de reutilização. Assim, valores elevados de WMC podem indicar classes com maior responsabilidade interna e maior esforço de compreensão.

Semelhantemente, Sommerville (2011) relaciona valores mais altos de WMC a classes mais complexas e potencialmente mais difíceis de compreender. Classes com muitos métodos ou métodos mais complexos podem concentrar diferentes responsabilidades, o que pode afetar sua coesão lógica e dificultar sua reutilização em hierarquias de herança ou em outros contextos do sistema.

No âmbito desta pesquisa, a WMC é utilizada como indicador relacionado à complexidade estrutural das classes. A análise dessa métrica permite observar se os sistemas estudados apresentam classes com maior concentração de comportamento ou maior complexidade interna, contribuindo para a comparação entre a organização estrutural da arquitetura em camadas e da arquitetura hexagonal.

### *2.4.5 Fan-In*

A métrica *Fan-In* está ligada à quantidade de elementos que direcionam dependências, chamadas ou fluxos para um determinado módulo, função, método ou componente. Em termos gerais, ela representa o número de partes do sistema que utilizam ou dependem de um elemento analisado, permitindo observar o grau de entrada de dependências sobre esse elemento.

Segundo Fenton e Bieman (2015), o *fan-in* de um módulo pode ser compreendido como a quantidade de fluxos locais que terminam nesse módulo, juntamente com as estruturas de dados das quais ele recupera informações. Embora essa definição esteja associada a uma visão modular mais geral, ela evidencia a ideia central da métrica: identificar quantos fluxos ou dependências chegam a uma unidade do sistema.

De forma semelhante, Sommerville (2011) define *fan-in* como uma medida do número de funções ou métodos que chamam uma determinada função ou método. Nesse sentido, valores elevados de *fan-in* indicam que o elemento analisado é utilizado por várias partes do sistema, podendo representar uma unidade central na estrutura do software. Entretanto, essa centralidade também implica maior atenção, pois alterações nesse elemento podem produzir efeitos em diferentes partes do sistema que dependem dele.

A interpretação do *Fan-In* não deve ser feita apenas de forma negativa. Yourdon e Constantine (1978) observam que, em determinados contextos, aumentar o *fan-in* pode ser desejável, pois indica que funcionalidades comuns estão sendo reutilizadas em vez de duplicadas em diferentes partes do sistema. Nesse sentido, o *Fan-In* pode estar associado à modularidade quando representa o reaproveitamento adequado de responsabilidades bem definidas.

Porém, Yourdon e Constantine (1978) também ressaltam que o aumento do *fan-in* não deve ocorrer a qualquer custo. Agrupar funcionalidades não relacionadas em um único módulo apenas para aumentar seu uso pode comprometer a coesão e gerar componentes excessivamente genéricos ou difíceis de manter. Assim, valores elevados de *Fan-In* devem ser analisados em conjunto com outras métricas, especialmente aquelas relacionadas à coesão e ao acoplamento.

Nesse sentido, o *Fan-In* é utilizado como indicador da quantidade de dependências que chegam a uma classe. A análise dessa métrica permite identificar classes mais centrais ou mais reutilizadas dentro da estrutura dos sistemas estudados. Dessa forma, o *Fan-In* contribui para compreender como as dependências se distribuem nas arquiteturas analisadas e quais classes concentram maior número de relações de entrada.

### *2.4.6 Fan-Out*

A métrica *Fan-Out* relaciona-se à quantidade de dependências, chamadas ou fluxos que saem de um determinado módulo, função, método ou componente. Enquanto o *Fan-In* observa quantos elementos utilizam uma unidade do sistema, o *Fan-Out* considera quantos elementos são utilizados por essa unidade. Dessa forma, a métrica permite analisar o grau em que um elemento depende de outros para executar suas responsabilidades.

Segundo Fenton e Bieman (2015), o *fan-out* de um módulo corresponde ao número de fluxos locais que partem desse módulo, juntamente com as estruturas de dados que são atualizadas por ele. Essa definição, embora apresentada em um contexto modular mais geral, evidencia a ideia central da métrica: identificar a quantidade de relações de saída estabelecidas por uma unidade do sistema.

De modo semelhante, Sommerville (2011) define *fan-out* como o número de funções chamadas por uma determinada função. Para o autor, valores elevados dessa métrica podem indicar maior complexidade, pois o elemento analisado precisa coordenar um número maior de componentes chamados. Assim, quanto maior o *Fan-Out*, maior pode ser a complexidade da lógica necessária para controlar ou organizar essas interações.

A relação entre *Fan-Out* e complexidade também é discutida por Yourdon e Constantine (1978), que associa essa métrica ao conceito de amplitude de controle de um módulo. Nessa perspectiva, o *Fan-Out* representa o número de subordinados imediatos de um módulo. Valores muito elevados podem indicar que um módulo foi decomposto de maneira inadequada ou que há ausência de níveis intermediários na estrutura do sistema, fazendo com que uma unidade concentre muitas relações diretas com outras partes.

No entanto, valores de *Fan-Out* devem ser interpretados de acordo com o contexto. Um valor mais alto pode ser justificado quando uma classe ou módulo possui uma responsabilidade legítima de coordenação. Por outro lado, quando esse valor indica dependência excessiva de muitos elementos distintos, pode sugerir maior acoplamento, maior complexidade de manutenção e maior dificuldade de compreensão da estrutura interna do sistema.

Dessa forma, o *Fan-Out* é utilizado como indicador da quantidade de dependências que partem de uma classe. A análise dessa métrica permite observar quais classes dependem de maior número de elementos externos para cumprir suas responsabilidades. Assim, o *Fan-Out* contribui para compreender a distribuição das dependências de saída nos sistemas estudados e complementa a análise realizada por meio das métricas CBO, RFC e *Fan-In*.

# 3 TRABALHOS RELACIONADOS

Este capítulo apresenta trabalhos relacionados à avaliação estrutural de sistemas de software e à comparação entre estilos arquiteturais. O levantamento considerou estudos que abordam métricas de software, modularidade, acoplamento, coesão, qualidade arquitetural e efeitos de decisões arquiteturais sobre atributos internos do sistema.

Os trabalhos foram identificados por meio de buscas no Google Acadêmico, no Portal de Periódicos da CAPES e na base IEEE Xplore, além da consulta a repositórios institucionais de trabalhos acadêmicos. Foram utilizadas combinações de termos em português e em inglês, como "métricas de software" e "*software metrics*", "modularidade" e "*modularity*", "arquitetura de software" e "*software architecture*", "arquitetura em camadas" e "*layered architecture*", "arquitetura hexagonal" e "*hexagonal architecture*", além de "sistemas orientados a objetos" e "*object-oriented systems*".

A seleção incluiu artigos científicos, dissertações e trabalhos de conclusão de curso. Essa decisão decorre da especificidade do recorte desta pesquisa, uma vez que, durante o levantamento realizado, não foram identificados, nas bases consultadas, artigos com o mesmo recorte comparativo entre arquitetura em camadas e arquitetura hexagonal sob a perspectiva da modularidade estrutural. Assim, foram selecionados também trabalhos acadêmicos com proximidade temática ou metodológica, especialmente aqueles relacionados à avaliação estrutural de software, comparação entre arquiteturas, uso de métricas ou análise de atributos associados à modularidade.

Os trabalhos analisados não possuem o mesmo recorte desta pesquisa, mas contribuem para situar a questão investigada. Alguns utilizam métricas estruturais para avaliar sistemas orientados a objetos, enquanto outros comparam arquiteturas ou discutem critérios para escolha arquitetural. Em conjunto, esses estudos permitem observar diferentes formas de relacionar arquitetura, modularidade e qualidade estrutural.

## 3.1 Avaliação de Conectividade em Sistemas Orientados por Objetos

O trabalho de Ferreira (2006) propõe o Modelo de Avaliação de Conectividade de Sistemas Orientados por Objetos (MACSOO), fundamentado na tese de que a conectividade entre módulos é o fator preponderante para determinar a estabilidade e a manutenibilidade de um software. A pesquisa parte do princípio de que a manutenção consome a maior parte do

custo total de um sistema e que estruturas mais flexíveis, com menor interdependência entre suas partes, facilitam esse processo.

Segundo a autora, a orientação por objetos, embora favoreça a modularidade, não garante por si só uma estrutura de alta qualidade se não houver um controle rigoroso sobre as conexões entre classes. O estudo defende que o impacto de uma alteração no sistema é diretamente proporcional ao seu grau de conectividade, e que o "apodrecimento" do *design* de software ocorre quando a rigidez estrutural impede modificações sem gerar efeitos colaterais em cascata.

Para operacionalizar a avaliação, o trabalho identifica e classifica métricas estruturais clássicas, com destaque para o conjunto CK (Chidamber e Kemerer) e o conjunto MOOD (*Metrics for Object Oriented Design*), selecionando indicadores como CBO (*Coupling Between Object Classes*), LCOM (*Lack of Cohesion in Methods*) e COF (*Coupling Factor*). Além disso, a pesquisa contribuiu com o desenvolvimento da ferramenta *Connecta*, capaz de extrair essas métricas de sistemas desenvolvidos em Java e aplicar uma adaptação do modelo de estabilidade de Myers para prever o esforço de manutenção.

O trabalho de Ferreira (2006) aproxima-se desta pesquisa por utilizar métricas estruturais para avaliar atributos internos de sistemas orientados a objetos, especialmente aqueles relacionados ao acoplamento e à coesão. No entanto, seu foco está na proposição de um modelo de avaliação de conectividade e de uma ferramenta de apoio à análise de sistemas Java. Esta pesquisa, por sua vez, utiliza métricas estruturais já estabelecidas para realizar uma comparação entre dois estilos arquiteturais específicos, arquitetura em camadas e arquitetura hexagonal, observando como esses estilos podem se refletir em indicadores de acoplamento, coesão e complexidade estrutural.

## 3.2 Os efeitos da arquitetura dirigida a eventos na modularidade de software

O trabalho de Lazzari e Farias (2021) apresenta um estudo que busca compreender os efeitos da arquitetura dirigida a eventos na modularização de software em comparação ao estilo arquitetural *REST*. A pesquisa fundamenta-se na análise de duas versões funcionalmente idênticas de uma aplicação real, evoluídas por meio de cinco cenários de mudança nos quais novas funcionalidades foram adicionadas. O estudo utiliza um conjunto de dez métricas estruturais para quantificar variáveis críticas da modularidade, incluindo separação de interesses (SoC), acoplamento, coesão, complexidade e tamanho.

Segundo os resultados reportados pelos autores, a arquitetura dirigida a eventos (implementada com Apache Kafka) demonstrou superioridade na separação de interesses, apresentando melhores índices de modularização funcional tanto em nível de componentes quanto de linhas de código. Esse achado sugere que o modelo de eventos favorece a independência dos serviços, facilitando a evolução do sistema sem a degradação da qualidade funcional. Por outro lado, a pesquisa revelou que essa flexibilidade introduz uma "penalidade" estrutural: a arquitetura dirigida a eventos apresentou maior acoplamento e complexidade se comparada ao *REST*, além de um aumento significativo no tamanho do código devido às necessidades de configuração de infraestrutura e classes auxiliares.

No estudo de Lazzari e Farias (2021), a comparação arquitetural é conduzida a partir da evolução de uma aplicação e do contraste entre *REST* e arquitetura dirigida a eventos. Esse trabalho é relevante para esta pesquisa por evidenciar que decisões arquiteturais podem produzir efeitos distintos sobre atributos de modularidade. Aqui, porém, o interesse não recai sobre o mecanismo de comunicação entre serviços, mas sobre a organização interna de sistemas orientados a objetos, analisada por meio de métricas em nível de classe.

## 3.3 Comparação entre as arquiteturas de software monolítica e de microsserviços

O trabalho de Vale (2025) investiga o debate entre as arquiteturas monolítica e de microsserviços, focando na transição da análise teórica para a validação empírica. O estudo busca mensurar os *trade-offs* de cada abordagem em um ambiente controlado. A metodologia consistiu na implementação de dois protótipos funcionalmente idênticos de um sistema de *e-commerce* (*CRUD*), desenvolvidos com o ecossistema Node.js e containerizados com Docker, com o objetivo de reduzir interferências externas na comparação entre as abordagens arquiteturais.

Segundo os resultados reportados pelo autor, a arquitetura monolítica demonstrou ser objetivamente mais eficiente em todas as métricas de performance local e custo operacional. O achado mais significativo revelou uma penalidade de desempenho de 147% na latência de operações que exigiam orquestração entre múltiplos domínios na arquitetura de microsserviços, em comparação ao monólito. Além disso, o estudo quantificou o "imposto de memória" da distribuição, apontando que os microsserviços consumiram 271% mais memória RAM em estado de repouso devido à sobrecarga de manter múltiplos processos e contêineres ativos.

No que tange à complexidade, o trabalho apresenta uma análise qualitativa indicando que a arquitetura de microsserviços impõe uma carga cognitiva e operacional superior. Isso foi

evidenciado pelo número de artefatos de configuração necessários para a implantação, que foi três vezes maior do que o exigido para o monólito. O autor conclui que, embora os microsserviços ofereçam benefícios teóricos de escalabilidade, para sistemas de baixa complexidade ou em fases iniciais, a arquitetura monolítica é fortemente suportada pela eficiência de recursos e velocidade de entrega.

O trabalho de Vale (2025) contribui para o debate sobre *trade-offs* arquiteturais ao analisar desempenho, consumo de recursos e complexidade de implantação em protótipos funcionalmente equivalentes. Esse recorte difere do adotado nesta pesquisa, que não avalia eficiência de execução nem custo operacional, mas características estruturais do código-fonte. Assim, a comparação realizada neste TCC concentra-se nos indicadores de acoplamento, coesão e complexidade das classes em uma arquitetura em camadas e em uma arquitetura hexagonal.

## 3.4 Análise comparativa entre arquiteturas de software: monolítica, modular e microsserviços

O trabalho de Cintra *et al.* (2025) realiza uma análise comparativa entre as arquiteturas monolítica, monólito modular e microsserviços, buscando identificar características, vantagens, limitações e cenários de aplicação de cada abordagem arquitetural. O estudo é fundamentado em revisão bibliográfica e análise qualitativa, considerando atributos relevantes da Engenharia de Software, como escalabilidade, manutenibilidade, resiliência e complexidade operacional.

Segundo os autores, a arquitetura monolítica apresenta vantagens relacionadas à simplicidade de desenvolvimento, implantação e gerenciamento, especialmente em sistemas de pequeno e médio porte. Entretanto, à medida que o sistema cresce, problemas relacionados ao aumento do acoplamento e à dificuldade de manutenção tornam-se mais evidentes. Em contrapartida, a arquitetura de microsserviços oferece maior flexibilidade, escalabilidade independente e isolamento de falhas, embora introduza desafios associados à comunicação distribuída, monitoramento e maior complexidade operacional (Cintra *et al.*, 2025).

O trabalho também destaca o monólito modular como uma alternativa intermediária entre os dois extremos arquiteturais. De acordo com os autores, essa abordagem busca manter a simplicidade estrutural do monólito tradicional, porém com maior organização interna e separação de responsabilidades, favorecendo a manutenibilidade e facilitando uma possível migração incremental para microsserviços. Além disso, o estudo propõe um guia heurístico para

apoio à tomada de decisão arquitetural, utilizando pesos e pontuações atribuídos a diferentes atributos de qualidade.

Como o estudo de Cintra *et al.* (2025) é baseado em revisão bibliográfica, sua contribuição está na sistematização das diferenças entre arquitetura monolítica, monólito modular e microsserviços, bem como na proposição de um guia de apoio à decisão arquitetural. Este TCC adota outro nível de análise: parte de sistemas implementados e utiliza métricas estruturais em nível de classe para comparar arquitetura em camadas e arquitetura hexagonal quanto a acoplamento, coesão e complexidade estrutural.

## 3.5 Um estudo sobre a relação entre qualidade e arquitetura de software

O trabalho de Tsuruta (2010) investiga como a arquitetura de software atua como fator determinante para a qualidade final do produto, analisando quatro abordagens distintas para sua especificação: clássica, orientada a objetos, orientada a atributos e orientada a busca. A pesquisa fundamenta-se na premissa de que a qualidade não reside apenas na ausência de defeitos funcionais, mas na capacidade do sistema de satisfazer requisitos não-funcionais críticos, como a manutenibilidade e a capacidade de modificação.

O autor destaca que o *design* arquitetural deve ser guiado pelos atributos de qualidade mais importantes para o sucesso do projeto. Para viabilizar essa análise, o estudo utiliza a métrica de arquitetura denominada Qualidade de Modularização (TurboMQ), que quantifica a interdependência entre componentes ao calcular a razão entre relacionamentos internos e externos em um gráfico de dependência de módulos (MDG). Um dos diferenciais da pesquisa é a aplicação da Engenharia de Software Baseada em Busca (SBSE), utilizando Algoritmos Genéticos para encontrar automaticamente o particionamento ideal de responsabilidades em subsistemas que maximize a modularidade.

A contribuição de Tsuruta (2010) está em relacionar qualidade de software e decisões arquiteturais, mostrando como a organização de responsabilidades pode ser tratada como um problema de modularização. Seu estudo, porém, utiliza a métrica TurboMQ em um contexto de otimização arquitetural apoiado por algoritmos genéticos. Este TCC segue outro caminho: em vez de buscar automaticamente uma melhor divisão em subsistemas, analisa sistemas já implementados e compara, por meio de métricas estruturais em nível de classe, como a arquitetura em camadas e a arquitetura hexagonal se refletem no acoplamento, na coesão e na complexidade do código-fonte.

## 3.6 Síntese e lacunas

Após a análise individual dos trabalhos, esta seção apresenta uma visão consolidada das pesquisas selecionadas. O Quadro 1 sistematiza os recortes dos estudos, as arquiteturas ou abordagens analisadas, as métricas ou atributos considerados e a relação de cada trabalho com esta pesquisa.

Quadro 1 – Síntese dos trabalhos relacionados

| **Autor (Ano)** | **Recorte do estudo** | **Arquiteturas ou abordagens analisadas** | **Métricas ou atributos considerados** | **Relação com esta pesquisa** |
|---|---|---|---|---|
| Ferreira (2006) | Avaliação de conectividade em sistemas orientados a objetos. | Sistemas OO desenvolvidos em Java. | Métricas estruturais associadas a acoplamento, coesão e conectividade. | Utiliza métricas estruturais, mas não compara estilos arquiteturais específicos. |
| Tsuruta (2010) | Relação entre arquitetura de software e atributos de qualidade. | Abordagens clássica, orientada a objetos, orientada a atributos e orientada a busca. | Qualidade de modularização, TurboMQ e particionamento de responsabilidades. | Relaciona arquitetura e qualidade, mas com foco em otimização arquitetural. |
| Lazzari (2021) | Efeitos arquiteturais sobre modularização em cenários de evolução. | Arquitetura dirigida a eventos e *REST*. | Separação de interesses, acoplamento, coesão, complexidade e tamanho. | Compara arquiteturas com métricas, mas analisa comunicação entre serviços. |
| Cintra et al. (2025) | Comparação conceitual para apoio à decisão arquitetural. | Monolítica, monólito modular e microsserviços. | Escalabilidade, manutenibilidade, resiliência, complexidade operacional e outros atributos de qualidade. | Apresenta visão comparativa ampla, sem medição estrutural em implementações concretas. |
| Vale (2025) | Avaliação de custos operacionais em arquiteturas distintas. | Monolítica e microsserviços. | Latência, consumo de recursos e complexidade de implantação. | Compara arquiteturas quantitativamente, mas com foco em desempenho e infraestrutura. |
| **Este trabalho** | **Avaliação estrutural da modularidade em sistemas orientados a objetos.** | **Arquitetura em camadas e arquitetura hexagonal.** | **CBO, LCOM*, RFC, WMC, *Fan-In* e *Fan-Out*.** | **Compara sistemas por meio de métricas estruturais em nível de classe.** |

Fonte: Elaborado pelo autor (2026).

A análise dos trabalhos relacionados mostra que métricas estruturais e comparações arquiteturais têm sido utilizadas sob diferentes perspectivas para investigar atributos de qualidade de software. Alguns estudos concentram-se na conectividade e na modularidade interna de sistemas orientados a objetos, enquanto outros analisam *trade-offs* entre arquiteturas distribuídas, estilos de comunicação ou modelos de organização arquitetural. Também se observa diferença quanto ao tipo de evidência utilizada: há trabalhos baseados em revisão bibliográfica, estudos com protótipos controlados e pesquisas que aplicam métricas diretamente em sistemas implementados.

Considerando os trabalhos analisados, percebe-se que a comparação quantitativa entre arquitetura em camadas e arquitetura hexagonal ainda é pouco explorada, especialmente quando o foco recai sobre métricas estruturais em nível de classe. As pesquisas mais próximas abordam conectividade, modularidade ou *trade-offs* arquiteturais, mas geralmente se concentram em outros recortes, como monólitos e microsserviços, *REST* e arquitetura dirigida a eventos, modelos de otimização arquitetural ou guias conceituais de apoio à decisão.

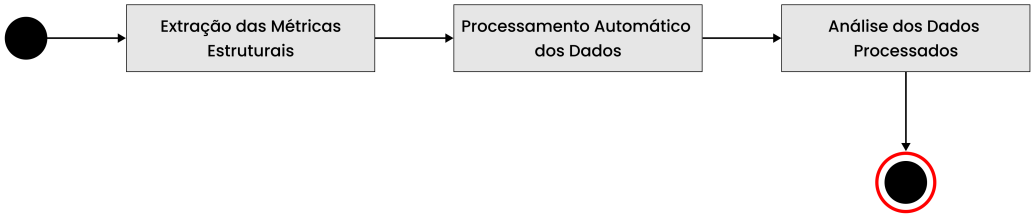
Nesse contexto, este TCC contribui com uma análise empírica descritiva de dois sistemas orientados a objetos desenvolvidos sob estilos arquiteturais distintos: SISO, organizado em camadas, e Roomie, organizado segundo a arquitetura hexagonal. A comparação não busca estabelecer conclusões gerais sobre a superioridade de um estilo arquitetural, mas observar como esses estilos podem se refletir em indicadores de acoplamento, coesão e complexidade estrutural extraídos do código-fonte.

# 4 METODOLOGIA

Este capítulo apresenta os procedimentos adotados para condução da pesquisa. Inicialmente, são caracterizados os sistemas analisados. Em seguida, é descrito o fluxo metodológico adotado, composto pela extração das métricas estruturais, processamento automatizado dos dados e análise dos dados processados. Por fim, é apresentado o cronograma de execução.

A Figura 3 apresenta uma visão geral do fluxo metodológico adotado nesta pesquisa.

Figura 3 – Fluxo metodológico da pesquisa




Fonte: Elaborado pelo autor (2026).

## 4.1 Objetos de Estudo

Esta pesquisa utiliza como objetos de estudo dois sistemas desenvolvidos em contexto acadêmico durante disciplinas do curso de Engenharia de Software. Os sistemas foram implementados por equipes de estudantes, com participação do autor desta pesquisa, e foram selecionados por adotarem estilos arquiteturais distintos, possibilitando a investigação dos impactos dessas arquiteturas sobre atributos relacionados à modularidade de software.

O primeiro sistema analisado é o *SISO*, desenvolvido na disciplina de Projeto Integrado II. Trata-se de uma aplicação web voltada ao gerenciamento de clínicas odontológicas, oferecendo funcionalidades relacionadas ao controle de usuários, cadastro de pacientes, prontuário eletrônico, anamnese, gerenciamento de tratamentos e agendamentos. O sistema foi concebido utilizando uma arquitetura em camadas, modelo no qual as responsabilidades da aplicação são distribuídas em níveis específicos, favorecendo a separação de interesses e a organização estrutural do software (Fowler, 2003).

O segundo sistema analisado é o *Roomie*, desenvolvido na disciplina de Projeto Integrado III. O sistema consiste em uma aplicação móvel destinada a conectar pessoas que possuem vagas disponíveis em residências com indivíduos que buscam compartilhar moradia. Entre suas funcionalidades estão a publicação de anúncios, mecanismos de filtragem, sistema de

compatibilidade entre usuários, troca de mensagens por *chat* e envio de notificações. Diferentemente do SISO, o Roomie foi estruturado segundo a arquitetura hexagonal, também conhecida como *Ports and Adapters*. De acordo com Cockburn (2005), esse estilo arquitetural busca isolar as regras de negócio dos detalhes tecnológicos por meio de interfaces bem definidas, reduzindo o acoplamento entre domínio e infraestrutura.

Embora os sistemas pertençam a domínios de negócio distintos, ambos foram desenvolvidos utilizando Java e o *framework* Spring Boot em suas camadas de *back-end*, adotando princípios de orientação a objetos e práticas semelhantes de desenvolvimento. Dessa forma, a comparação proposta concentra-se na organização estrutural promovida pelos estilos arquiteturais adotados, e não nas funcionalidades implementadas ou no desempenho das aplicações.

Tabela 1 – Caracterização dos sistemas analisados

| **Característica** | **SISO** | **Roomie** |
|---|---|---|
| Arquitetura | Em Camadas | Hexagonal |
| Classes | 89 | 88 |
| Interfaces | 47 | 38 |
| Enums | 6 | 18 |

Fonte: Elaborado pelo autor (2026).

Como observa-se na Tabela 1, os sistemas apresentam quantidades semelhantes de classes, indicando porte estrutural comparável sob essa perspectiva. Essa característica contribui para a realização de uma análise comparativa focada nos efeitos dos estilos arquiteturais sobre métricas relacionadas à modularidade, reduzindo a influência de diferenças significativas de escala entre os projetos. As quantidades de interfaces e enums são apresentadas apenas para caracterização geral dos sistemas, enquanto a análise das métricas estruturais será conduzida considerando exclusivamente as classes extraídas pela ferramenta.

Os repositórios dos sistemas analisados encontram-se disponíveis para consulta e reprodução dos experimentos realizados neste trabalho: SISO[1] e Roomie[2].

## 4.2 Fluxo Metodológico

As etapas representadas na Figura 3 descrevem o procedimento adotado para obtenção e análise das métricas estruturais dos sistemas estudados. O fluxo inicia-se a partir dos dois objetos de estudo definidos nesta pesquisa, SISO e Roomie, e segue com a extração das métricas,

[1] https://github.com/Werley-08/SISO

[2] https://github.com/WendelRodriguesz/Roomie_Android

o processamento automatizado dos dados e a análise dos dados processados.

### *4.2.1 Extração das Métricas Estruturais*

A primeira etapa consiste na extração das métricas estruturais a partir do código-fonte dos sistemas selecionados. A coleta será realizada por meio de análise estática, abordagem na qual as métricas são obtidas diretamente a partir da estrutura do software, sem a necessidade de executar as aplicações (Sommerville, 2011). Essa estratégia é adequada ao objetivo desta pesquisa, uma vez que o estudo se concentra em atributos internos relacionados à organização estrutural dos sistemas.

Para a extração das métricas será utilizada a ferramenta CK, desenvolvida por Maurício Aniche. A ferramenta realiza análise estática de projetos Java e gera relatórios contendo métricas em nível de classe e de método (Aniche, 2015). Nesta pesquisa, serão utilizados exclusivamente os dados extraídos em nível de classe, pois o foco do estudo está na organização estrutural dos sistemas e não na complexidade individual de métodos específicos. A ferramenta CK[3] encontra-se disponível para consulta em seu repositório público.

A escolha da ferramenta CK justifica-se por sua compatibilidade com o contexto desta pesquisa e por sua adoção em estudos recentes que utilizam métricas extraídas do código-fonte. Arab *et al.* (2025), por exemplo, utilizaram a ferramenta CK para extrair métricas estáticas em nível de classe em projetos Java. De forma semelhante, Jin *et al.* (2023) empregaram o CK, juntamente ao SonarQube, para extrair métricas de repositórios de software livre. Assim, a ferramenta foi adotada neste trabalho por permitir a extração padronizada das métricas selecionadas nos dois sistemas analisados.

A unidade de análise corresponde aos registros em nível de classe gerados pela ferramenta CK. Esse nível de análise é coerente com o foco da pesquisa em sistemas orientados a objetos, uma vez que parte das métricas selecionadas foi concebida para avaliação de características estruturais de classes (Chidamber; Kemerer, 1994). Além disso, trabalhos relacionados analisados neste estudo utilizam métricas estruturais como instrumento para avaliação de modularidade e qualidade arquitetural, seja por meio de métricas orientadas a objetos (Ferreira, 2006), seja por abordagens quantitativas aplicadas à comparação entre estilos arquiteturais (Lazzari; Farias, 2021; Tsuruta, 2010).

As métricas utilizadas foram selecionadas com base em sua relação com atributos

[3] https://github.com/mauricioaniche/ck

associados à modularidade e à qualidade estrutural de sistemas orientados a objetos, especialmente acoplamento, coesão e complexidade. Parte dessas métricas deriva da suíte CK, proposta por Chidamber e Kemerer (1994), enquanto *Fan-In* e *Fan-Out* são incorporadas como métricas complementares para análise das dependências estruturais entre classes (Yourdon; Constantine, 1978; Sommerville, 2011).

Embora a suíte CK original seja composta por seis métricas, este trabalho utiliza CBO, LCOM, RFC e WMC, excluindo DIT e NOC. Essa decisão justifica-se pelo fato de DIT e NOC estarem mais diretamente relacionadas à profundidade e à estrutura de hierarquias de herança. Como o foco desta pesquisa está na modularidade estrutural, nas dependências entre classes e na separação de responsabilidades promovida pelos estilos arquiteturais analisados, métricas centradas em herança não foram priorizadas.

Para complementar a análise, foram incorporadas as métricas *Fan-In* e *Fan-Out*. Essas métricas permitem observar, respectivamente, a quantidade de dependências recebidas por uma classe e a quantidade de dependências realizadas por uma classe (Yourdon; Constantine, 1978; Sommerville, 2011). Dessa forma, elas contribuem para a análise do acoplamento estrutural ao evidenciar a direção e a concentração das dependências entre os componentes dos sistemas estudados. As métricas selecionadas para a análise são descritas no Quadro 2.

Quadro 2 – Métricas utilizadas na avaliação da modularidade estrutural dos sistemas

| **Métrica** | **Descrição** |
|---|---|
| CBO | Conta o número de dependências de uma classe. Na ferramenta CK, essas dependências são identificadas a partir dos tipos utilizados ao longo da classe, incluindo declarações de atributos, tipos de retorno de métodos, parâmetros e declarações de variáveis. Dependências associadas à própria biblioteca Java, como `java.lang.String`, são desconsideradas (Aniche, 2015). |
| LCOM* | Indica a falta de coesão entre os métodos de uma classe. Na ferramenta CK, é utilizada uma versão normalizada da métrica, com valores no intervalo de 0 a 1. Valores próximos de 0 indicam maior coesão, enquanto valores próximos de 1 indicam menor coesão, conforme a definição operacional adotada pela ferramenta (Aniche, 2015). |
| RFC | Indica o conjunto de respostas potenciais associado a uma classe. Na ferramenta CK, essa métrica é operacionalizada pela contagem de invocações únicas de métodos identificadas na classe por meio de análise estática (Aniche, 2015). |
| WMC | Indica a complexidade de uma classe. Na ferramenta CK, é calculado como o número de instruções de ramificação presentes nos métodos da classe (Aniche, 2015). |
| *Fan-In* | Conta o número de dependências de entrada de uma classe. Na ferramenta CK, corresponde ao número de classes que referenciam a classe analisada, seja por meio de atributos, acesso a atributos, invocação de métodos ou outras formas de referência identificadas por análise estática (Aniche, 2015). |
| *Fan-Out* | Conta o número de dependências de saída de uma classe. Na ferramenta CK, corresponde ao número de outras classes referenciadas pela classe analisada, seja por meio de atributos, invocações de métodos, instanciação de objetos ou outras formas de referência identificadas por análise estática (Aniche, 2015). |

Fonte: Elaborado pelo autor (2026).

Convém observar que as definições apresentadas no Quadro 2 apresentam variações em relação às proposições conceituais originais de Chidamber e Kemerer (1994). Tais distinções decorrem da operacionalização prática adotada pela ferramenta CK, que utiliza heurísticas de análise estática específicas para o ecossistema Java. Por exemplo, enquanto o CBO original é definido de forma ampla como o acoplamento entre classes, a ferramenta especifica exatamente quais referências (como variáveis locais e retornos) são contabilizadas e ignora propositalmente as bibliotecas padrão do Java para garantir maior precisão nos resultados. Da mesma forma, a métrica de coesão utilizada é o LCOM*, uma versão normalizada que difere do cálculo de

interseção de conjuntos proposto inicialmente.

Para a métrica RFC, a documentação da ferramenta CK indica uma limitação relacionada à resolução estática de invocações em métodos sobrecarregados com a mesma quantidade de parâmetros e tipos diferentes.

Para garantir consistência na comparação, as mesmas definições operacionais, a mesma ferramenta e os mesmos procedimentos de extração serão aplicados aos dois sistemas analisados. Dessa forma, ainda que existam variações entre definições conceituais e implementações práticas das métricas, a análise comparativa será conduzida com base em critérios uniformes de mensuração.

### *4.2.2 Processamento Automatizado dos Dados*

Após a extração das métricas pela ferramenta CK, os relatórios gerados em formato *CSV* (*Comma-Separated Values*) serão utilizados como entrada para um *script* desenvolvido em Python. Esse *script* será responsável por ler os arquivos de saída da ferramenta, selecionar as métricas definidas nesta pesquisa e calcular as medidas estatísticas utilizadas na análise comparativa.

Para cada sistema e para cada métrica selecionada, serão calculadas medidas de estatística descritiva, incluindo média, mediana, desvio padrão, valor mínimo e valor máximo. Essas medidas serão utilizadas para sumarizar a distribuição dos valores observados em cada métrica. A média será considerada como uma medida geral de tendência central, enquanto a mediana será analisada em conjunto, especialmente em distribuições que apresentem assimetria ou valores extremos. O desvio padrão será utilizado como indicador da dispersão dos valores em torno da média, e os valores mínimo e máximo permitirão observar a amplitude dos resultados obtidos. Essa escolha considera que dados de medição de software podem apresentar distribuições não normais, assimétricas e com valores extremos, exigindo cautela na interpretação das medidas estatísticas (Fenton; Bieman, 2015).

Além do cálculo das medidas descritivas, o *script* também será responsável pela geração de visualizações gráficas que auxiliem a comparação entre os sistemas. Essas visualizações terão como objetivo representar a distribuição dos valores das métricas e facilitar a identificação de diferenças, concentrações e possíveis valores extremos.

A utilização de um *script* para leitura, processamento e visualização dos dados busca reduzir intervenções manuais sobre os dados gerados pela ferramenta CK, diminuindo

a possibilidade de erros de manipulação e favorecendo a reprodutibilidade dos procedimentos adotados. Não serão aplicados testes estatísticos inferenciais, pois o objetivo desta pesquisa é descrever e comparar os indicadores estruturais observados nos dois sistemas analisados.

### *4.2.3 Análise dos Dados*

A etapa final consiste na análise comparativa e na interpretação dos valores obtidos para os sistemas SISO e Roomie. A comparação será realizada métrica por métrica, considerando as medidas descritivas calculadas e as visualizações gráficas geradas pelo *script* em Python.

A análise buscará identificar semelhanças e diferenças nos indicadores estruturais associados à modularidade, observando a concentração dos dados, a variação entre as classes e a ocorrência de possíveis valores extremos. As métricas CBO, *Fan-In* e *Fan-Out* serão consideradas na análise das dependências entre classes; a métrica LCOM* será utilizada como indicador relacionado à coesão interna; e as métricas RFC e WMC serão consideradas na análise da complexidade estrutural e do potencial de interação das classes.

Os resultados serão interpretados com base nos conceitos discutidos na fundamentação teórica, especialmente modularidade, acoplamento, coesão e complexidade. Nesse sentido, será observado como a arquitetura em camadas adotada no SISO e a arquitetura hexagonal adotada no Roomie se refletem nos indicadores estruturais extraídos.

A comparação não tem como objetivo avaliar qual sistema é funcionalmente melhor ou mais eficiente em termos de desempenho. Os valores obtidos serão interpretados como indicadores quantitativos relacionados à organização estrutural dos sistemas, e não como medidas absolutas de qualidade arquitetural.

A interpretação dos resultados considerará o escopo da pesquisa, uma vez que os sistemas analisados pertencem a domínios distintos e foram desenvolvidos em contexto acadêmico. Dessa forma, os resultados serão discutidos como evidências observadas nos objetos de estudo analisados, contribuindo para a compreensão de como diferentes estilos arquiteturais podem se refletir em métricas estruturais de sistemas orientados a objetos.

## 4.3 Cronograma de Execução

Esta seção apresenta o cronograma de execução das atividades previstas para a realização do TCC II, conforme descrito na Tabela 2.

Tabela 2 – Cronograma de execução do TCC II

| **Atividade** | **Ago** | **Set** | **Out** | **Nov** | **Dez** |
|---|---|---|---|---|---|
| Revisão e ajustes do TCC I | X | | | | |
| Extração das métricas com a ferramenta CK | | X | | | |
| Desenvolvimento do *script* de análise | | X | X | | |
| Processamento dos dados | | | X | | |
| Análise dos dados processados | | | X | X | |
| Redação dos resultados e discussão | | | | X | |
| Elaboração da conclusão | | | | X | |
| Revisão geral do trabalho | | | | X | X |
| Adequação às normas da instituição | | | | | X |
| Entrega da versão final | | | | | X |

Fonte: Elaborado pelo autor (2026).

# REFERÊNCIAS BIBLIOGRÁFICAS


ANICHE, M. **Java code metrics calculator (CK)**. [*S. l.*], 2015. Available in https://github.com/mauricioaniche/ck/.

ARAB, I.; MAGEL, K.; AKOUR, M. Evaluating the predictive power of software metrics for fault localization. **Computers**, v. 14, n. 6, 2025. ISSN 2073-431X. Disponível em: https://www.mdpi.com/2073-431X/14/6/222.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 3. ed. [*S. l.*]: Addison-Wesley, 2012.

CHIDAMBER, S. R.; KEMERER, C. F. A metrics suite for object oriented design. **IEEE Transactions on Software Engineering**, v. 20, n. 6, p. 476–493, 1994.

CINTRA, L. R.; MARTINIANO, P. P.; BORGES, L. Análise comparativa entre arquiteturas de software monolítica, modular e microsserviços. **RESIGET**, v. 15, n. 1, p. 180–209, 2025.

COCKBURN, A. **Hexagonal Architecture**. 2005. Acesso em: 24 abr. 2026. Disponível em: https://alistair.cockburn.us/hexagonal-architecture/.

FENTON, N.; BIEMAN, J. **Software Metrics: A Rigorous and Practical Approach**. 3. ed. [*S. l.*]: CRC Press, 2015.

FERREIRA, K. A. M. **Avaliação de Conectividade em Sistemas Orientados por Objetos**. Dissertação (Mestrado em Ciência da Computação) – Universidade Federal de Minas Gerais, Belo Horizonte, 2006.

FOWLER, M. **Patterns of Enterprise Application Architecture**. [*S. l.*]: Addison-Wesley, 2003.

JIN, S.; LI, Z.; CHEN, B.; ZHU, B.; XIA, Y. Software code quality measurement: Implications from metric distributions. In: **2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)**. [*S. l.*: *s. n.*], 2023. p. 488–496.

LAZZARI, L.; FARIAS, K. Event-driven architecture and REST: an exploratory study on modularity. **CoRR**, abs/2110.14699, 2021. Disponível em: https://arxiv.org/abs/2110.14699.

PARNAS, D. L. On the criteria to be used in decomposing systems into modules. **Communications of the ACM**, v. 15, n. 12, p. 1053–1058, 1972.

PRESSMAN, R. S. **Software Engineering: A Practitioner's Approach**. 7. ed. [*S. l.*]: McGraw-Hill, 2010.

SOMMERVILLE, I. **Software Engineering**. 9. ed. [*S. l.*]: Pearson, 2011.

TSURUTA, M. **Um estudo sobre a relação entre qualidade e arquitetura de software**. Dissertação (Mestrado em Administração) – Universidade de São Paulo, São Paulo, 2010.

VALE, P. T. P. do. **Comparação entre as arquiteturas de software monolítica e de microsserviços**. Monografia (Trabalho de Conclusão de Curso (Graduação em Engenharia da Computação)) – Universidade Federal do Ceará, Sobral, 2025.

YOURDON, E.; CONSTANTINE, L. L. **Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design**. [*S. l.*]: Yourdon Press, 1978.